Report - Version 4: Planner

Introduction

Version 4 introduces the Planner.

In the previous version, the project introduced the Generic Parser. The agent was already able to select a tool, extract the required arguments, execute the selected tool, and store the interaction in memory.

Version 4 keeps that structure, but adds one new step before parsing and execution: the agent now creates a simple execution plan.

This version is still rule-based. No LLM is used and no external framework is introduced.

Goal of Version 4

The main goal of Version 4 is to introduce planning.

The agent should not execute immediately after selecting a tool. It should first create a small plan that describes what it is going to do.

The plan contains three pieces of information:

- the selected tool;
- the reason for using that tool;
- the arguments required by the tool.

The important idea is simple: the agent starts to reason before acting.

What Changes from Version 3

Version 3 focused on the Parser. The Parser became generic and parameter-based.

Version 4 does not change the Parser as the main concept. The Parser still reads the parameters required by the selected tool and extracts the needed arguments.

The new component is the Planner.

The flow changes from:

```
User Request -> Router -> Generic Parser -> Executor -> Memory
```

to:

```
User Request -> Router -> Planner -> Generic Parser -> Executor -> Memory
```

The Router still selects the most suitable tool. The Planner then builds a small execution plan around that selected tool.

New Concept: Planner

The Planner is a new component that creates a plan before the agent parses arguments and executes a tool.

In this version, the Planner is simple and rule-based. It uses the existing Router to choose the most suitable tool.

If a suitable tool is found, the Planner returns a plan with the selected tool, the reason for using it, and the parameters required by the tool.

Example:

```
{
   "tool": "addition",
   "reason": "Add two numbers.",
   "needs_arguments": ["a", "b"]
}
```

This plan is then stored inside the agent state.

Unsupported Requests

The Planner also handles unsupported requests.

For example, the request:

```
agent("What is the weather in Rome?")
```

does not match any available tool.

In this case, the Planner returns:

```
{
   "tool": None,
   "reason": "No suitable tool was found for the request.",
   "needs_arguments": []
}
```

The Agent then returns a safe fallback result:

```
"I cannot handle this request yet."
```

This is important because the agent should fail clearly and safely when a request is outside its current capabilities.

Full Architecture in Version 4

The full flow of Version 4 is:

```
User Request -> Router -> Planner -> Generic Parser -> Executor -> Memory
↓
Output
```

The main responsibilities are now clearer:

```
Router = selects the tool
Planner = creates the execution plan
Parser = extracts the arguments
```

```
Executor = validates and executes
Agent = coordinates
Memory = stores the state
```

The Agent is still the coordinator. It now coordinates planning, parsing, execution, and memory storage.

Agent State in Version 4

The agent state now includes a new key: plan.

A typical state looks like this:

```
{
   "request": "What is 2 + 3?",
   "plan": {
      "tool": "addition",
      "reason": "Add two numbers.",
      "needs_arguments": ["a", "b"]
   },
   "action": "addition",
   "arguments": {"a": 2, "b": 3},
   "result": 5
}
```

This makes the internal behavior of the agent easier to inspect.

Tests

The tests verify that the agent continues to work after adding the Planner.

The main test cases are:

```
agent("What is 2 + 3?")
agent("Hello, what is 2 + 3?")
agent("Hello, my name is luca.")
agent("What is the weather in Rome?")
memory
```

These tests verify that:

- the Router still prioritizes the main intent;
- the Planner creates a plan before execution;
- the Parser still extracts the correct arguments;
- the Executor still validates and runs the selected tool;
- unsupported requests are handled with no selected tool;
- the Memory stores the plan together with the rest of the state.

What Version 4 Teaches

Version 4 teaches an important agent design principle:

> an agent should plan before acting

The Planner is still simple, but it introduces the idea that an agent can create an intermediate representation before execution.

This makes the architecture easier to extend in the future, especially when the project moves toward more advanced planners and multi-step agents.

Limitations

Version 4 improves the architecture, but some limitations remain:

- the Router is still rule-based;
- the Planner is still rule-based;
- the Parser still uses manual extraction rules;
- memory is still a simple list;
- there is no Tool Registry yet;
- there is no LLM yet;
- the agent still performs only one action at a time.

These limitations are expected and will be addressed gradually in later versions.

Conclusion

Version 4 is a natural evolution of Version 3.

The Generic Parser remains in place, but the new focus is the Planner.

The agent now creates a small execution plan before parsing arguments and executing the selected tool.

This makes the agent more transparent and prepares the project for more advanced agent behavior.

The next step will be Version 5, where the project will introduce a Memory Manager.